Lab Handout 4-F

1-D CNN over Feature Sequences and the Clean→Contended Operating-Condition Shift Experiment

Capstone — Module 4: AI-Driven Classification Pipeline

1 Objective

Build and train a 1-D convolutional network that consumes short *sequences* of feature vectors (rather than single vectors like the MLP in LH4-E), then run the defining operating-condition shift experiment: train on the clean regime (Module 3 campaigns C1, C2, and C4: single-station Target A only, DATA-only) and evaluate on the contended regime (C5: cooperative-target ping-pong). Both regimes come from the same hardware — this is a within-hardware distribution-shift study, the most honest test of generalisation the framework can produce. The accuracy gap is the framework's capstone machine-learning result.

2 Prerequisites

- Lab Handout 4-E complete (`mlp_manifest.json` present in `Code/AIML/` for the comparison column).
- LH4-B preprocessed artifacts present in `Code/AIML/artifacts/<task>/`.
- LH3-H merge produced `merged_dataset.csv` with `regime` column tagging each row as `clean` (C1, C2, C4) or `contended` (C5).

Op-shift validity is task-dependent

The operating-condition shift experiment is only structurally meaningful when both regimes contain rows of every label class. In the current C1+C2+C5 dataset:

- **SF (the canonical op-shift task):** SF7/8/9/10 all appear in clean (C2 sweep). SF7 also appears in contended (C5 uses SF7). Clean→contended is meaningful for SF7 rows; predictions on SF8/9/10 contended rows would be vacuous because there are none.
- **Modulation:** only LoRa is collected — the task is single-class and the script skips it.
- **Emitter ID:** Beacon ID 0x02 (Target B) appears *only* in contended (C5b). A clean-trained model has never seen the Target B class. Clean→contended accuracy on Target B rows is by construction zero. The op-shift number for emitter ID is therefore an upper bound on what label-uninformed retraining could fix, not a continuous distribution-shift measurement.
- **Packet type:** ACK (`pkt_type`=1) appears *only* in contended (C5b). Same caveat as emitter ID: clean→contended on ACK rows is identically zero.

SF is the task where the experiment lands as designed; report all four for completeness but interpret accordingly.

3 Background: Why a 1-D CNN on Tabular Features?

A single feature vector captures one instant. A short window of consecutive vectors — say the last eight packets from the same station — captures the *temporal texture* of the transmission: burstiness, inter-arrival regularity, RSSI drift, SNR stability, the cadence difference between DATA and ACK rows. A 1-D CNN over that sequence axis can learn filters that respond to this temporal structure. The MLP cannot, because it sees one vector at a time.

Sequence Construction

For each row r in the dataset, build an $8 \times F$ input by taking the eight most recent packets with the same `beacon_id` (or fewer if eight are not yet available; left-pad with zeros). This produces one sequence per row while preserving the label on r. The sequences are *within-run* only — do not straddle `run_id` boundaries, because that would force the model to learn transitions between very different channel conditions.

4 Procedure

4.1 Step 1 — Logging setup and sliding-window sequence build

`Code/AIML/cnn_train.py` follows the same logging pattern as LH4-A through LH4-E. Sequences are constructed by re-applying the LH4-B preprocessor (which already drops zero-variance columns and scales the features) so the per-row vector going into the CNN matches the per-row vector the MLP sees, just stacked into a length-8 history.

```python
import json
import logging
import sys
import time
from datetime import datetime, timezone
from math import sqrt
from pathlib import Path

import joblib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from scipy.stats import norm
from torch.utils.data import DataLoader, TensorDataset

SCRIPT_DIR    = Path(__file__).parent
LOG_DIR       = SCRIPT_DIR / "Logs"
ARTIFACTS_DIR = SCRIPT_DIR / "artifacts"
FIGS_DIR      = SCRIPT_DIR / "figs"
MANIFEST_PATH = SCRIPT_DIR / "cnn_manifest.json"
MERGED_CSV    = SCRIPT_DIR.parent / "Tools" / "CampaignCollect" / "
    merged_dataset.csv"

LOG_DIR.mkdir(parents=True, exist_ok=True)
FIGS_DIR.mkdir(parents=True, exist_ok=True)
```

```python

run_stamp   = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%SZ")
log_archive = LOG_DIR / f"cnn_train_{run_stamp}.log"
log_latest  = LOG_DIR / "cnn_train_latest.log"

log = logging.getLogger("cnn_train")
log.setLevel(logging.DEBUG)
log.handlers.clear()
fmt = logging.Formatter("%(asctime)s [%(levelname)s] %(message)s",
                        datefmt="%Y-%m-%d %H:%M:%S")
for h in (logging.StreamHandler(sys.stdout),
          logging.FileHandler(log_archive, mode="w", encoding="utf-8"),
          logging.FileHandler(log_latest,  mode="w", encoding="utf-8")):
    h.setFormatter(fmt)
    log.addHandler(h)

log.info("=== cnn_train.py starting ===")
log.info(f"Run stamp: {run_stamp}")

WINDOW = 8
LABEL_COL = {"sf": "label_sf", "mod": "label_mod",
             "beacon": "label_beacon", "pkt": "label_pkt"}
```

Listing 1: Top of `cnn_train.py` — imports, paths, logging.

```python
def build_sequences(task):
    df = pd.read_csv(MERGED_CSV)
    # Match LH4-B's join-artifact drop so the scaler input distribution
    # matches what the preprocessor was fit on.
    df = df[df["inter_arrival_ms"] > 0].reset_index(drop=True)

    pre = joblib.load(ARTIFACTS_DIR / task / "preprocessor.pkl")
    feats = pre["feature_cols"]    # already excludes zero-variance cols
    X_all = pre["scaler"].transform(df[feats].to_numpy(dtype=np.float32))

    classes = pre["encoder"].classes_
    raw_labels = df[LABEL_COL[task]].to_numpy()
    known = np.isin(raw_labels, classes)
    if (~known).any():
        log.warning(f"  {task}: {int((~known).sum())} rows have labels "
                    f"not seen at fit time; dropping")
    df = df[known].reset_index(drop=True)
    X_all = X_all[known]
    y_all = pre["encoder"].transform(df[LABEL_COL[task]])

    # Group by (run_id, beacon_id) so a per-station window is a true
    # per-station history (Target B sequences in C5b don't include
    # Target A packets even though they share a run_id).
    seqs, labels, splits, regimes = [], [], [], []
    for (_, _), grp in df.groupby(["run_id", "beacon_id"], sort=False):
        idxs = grp.index.to_numpy()
        for i, r in enumerate(idxs):
            lo = max(0, i - WINDOW + 1)
            win_idxs = idxs[lo:i + 1]
```

```
            window = X_all[win_idxs]
            if len(window) < WINDOW:
                pad = np.zeros((WINDOW - len(window), window.shape[1]),
                               dtype=np.float32)
                window = np.concatenate([pad, window], axis=0)
            seqs.append(window)
            labels.append(y_all[r])
            splits.append(df.at[r, "split"])
            regimes.append(df.at[r, "regime"])

    seqs    = np.stack(seqs).astype(np.float32)
    labels  = np.asarray(labels, dtype=np.int64)
    splits  = np.asarray(splits)
    regimes = np.asarray(regimes)
    for name in ["train", "val", "test"]:
        mask = splits == name
        np.save(ARTIFACTS_DIR / task / f"Xs_{name}.npy",   seqs[mask])
        np.save(ARTIFACTS_DIR / task / f"ys_{name}.npy",   labels[mask])
        np.save(ARTIFACTS_DIR / task / f"regs_{name}.npy", regimes[mask])
        log.info(f"  {task} {name}: {mask.sum()} sequences of shape "
                 f"{seqs.shape[1:]}; "
                 f"clean={(regimes[mask]=='clean').sum()}, "
                 f"contended={(regimes[mask]=='contended').sum()}")
    return classes
```

Listing 2: Sequence construction grouped by (run_id, beacon_id) to keep per-station histories clean.

4.2 Step 2 — CNN-1D model and canonical training

```
class CNN1D(nn.Module):
    def __init__(self, n_features, n_classes):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv1d(n_features, 16, kernel_size=3, padding=1), nn.ReLU(),
            nn.Conv1d(16,         32, kernel_size=3, padding=1), nn.ReLU(),
            nn.AdaptiveAvgPool1d(1),
            nn.Flatten(),
            nn.Linear(32, 32), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(32, n_classes),
        )

    def forward(self, x):
        # Input arrives as (B, T=8, F); Conv1d expects (B, F, T).
        return self.net(x.transpose(1, 2))


DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")


def wilson_ci(n_correct, n_total, alpha=0.05):
```

```python
    if n_total == 0:
        return 0.0, 1.0
    p = n_correct / n_total
    z = norm.ppf(1 - alpha / 2)
    denom = 1 + z * z / n_total
    center = (p + z * z / (2 * n_total)) / denom
    half = z * sqrt(p * (1 - p) / n_total
                    + z * z / (4 * n_total * n_total)) / denom
    return max(0.0, center - half), min(1.0, center + half)


def train_cnn(Xs_tr, ys_tr, Xs_va, ys_va, n_classes, seed=42,
              epochs=200, patience=15, lr=1e-3, batch_size=32):
    torch.manual_seed(seed); np.random.seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
    model = CNN1D(Xs_tr.shape[2], n_classes).to(DEVICE)
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    loss_fn = nn.CrossEntropyLoss()
    ds = TensorDataset(torch.from_numpy(Xs_tr).float(),
                       torch.from_numpy(ys_tr).long())
    ld = DataLoader(ds, batch_size=batch_size, shuffle=True)
    if len(Xs_va):
        X_va_t = torch.from_numpy(Xs_va).float().to(DEVICE)
        y_va_t = torch.from_numpy(ys_va).long().to(DEVICE)
    else:
        X_va_t = y_va_t = None

    best_val, best_state, bad = float("inf"), None, 0
    history = {"train_loss": [], "val_loss": []}
    for ep in range(epochs):
        model.train()
        total = 0.0
        for xb, yb in ld:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            opt.zero_grad()
            loss = loss_fn(model(xb), yb)
            loss.backward(); opt.step()
            total += loss.item() * len(xb)
        history["train_loss"].append(total / max(len(ds), 1))
        if X_va_t is not None:
            model.eval()
            with torch.no_grad():
                v_loss = loss_fn(model(X_va_t), y_va_t).item()
            history["val_loss"].append(v_loss)
            if v_loss < best_val - 1e-4:
                best_val, bad = v_loss, 0
                best_state = {k: v.clone() for k, v in model.state_dict().
                    items()}
            else:
                bad += 1
                if bad >= patience:
                    break
        else:
```

```
            best_state = {k: v.clone() for k, v in model.state_dict().items()}
    if best_state is not None:
        model.load_state_dict(best_state)
    return model, history


@torch.no_grad()
def accuracy(model, Xs, ys):
    if len(Xs) == 0:
        return float("nan"), 0, 0
    model.eval()
    xb = torch.from_numpy(Xs).float().to(DEVICE)
    pred = model(xb).argmax(1).cpu().numpy()
    n_correct = int((pred == ys).sum())
    return n_correct / len(ys), n_correct, len(ys)


# Canonical training run (seed 42) --- standard train/val/test pipeline.
results = {}
for task in ["sf", "mod", "beacon", "pkt"]:
    log.info(f"--- Task: {task} ---")
    pre = joblib.load(ARTIFACTS_DIR / task / "preprocessor.pkl")
    n_classes = len(pre["encoder"].classes_)
    if n_classes < 2:
        log.warning(f"  {task}: only {n_classes} class --- "
                    f"skipping CNN (cross-entropy degenerate for one class)")
        results[task] = {"skipped": True, "reason": "single class in train"}
        continue

    log.info(f"  Building sequences (window={WINDOW})...")
    build_sequences(task)
    Xs_tr = np.load(ARTIFACTS_DIR / task / "Xs_train.npy")
    ys_tr = np.load(ARTIFACTS_DIR / task / "ys_train.npy")
    Xs_va = np.load(ARTIFACTS_DIR / task / "Xs_val.npy")
    ys_va = np.load(ARTIFACTS_DIR / task / "ys_val.npy")
    Xs_te = np.load(ARTIFACTS_DIR / task / "Xs_test.npy")
    ys_te = np.load(ARTIFACTS_DIR / task / "ys_test.npy")

    t0 = time.time()
    model, history = train_cnn(Xs_tr, ys_tr, Xs_va, ys_va, n_classes)
    elapsed = time.time() - t0
    acc_te, nc, nt = accuracy(model, Xs_te, ys_te)
    lo, hi = wilson_ci(nc, nt)
    log.info(f"  CNN test acc = {acc_te:.3f}  [95% CI {lo:.3f}, {hi:.3f}]"
             f"({nc}/{nt})  trained in {elapsed:.1f}s, "
             f"epochs={len(history['train_loss'])}")
    torch.save({"state_dict": model.state_dict(),
                "n_features": Xs_tr.shape[2], "n_classes": n_classes,
                "seed": 42, "test_acc": acc_te, "ci": [lo, hi],
                "elapsed_s": elapsed, "history": history},
```

```
                ARTIFACTS_DIR / task / "cnn.pt")
    results[task] = {
        "test_acc": acc_te, "ci_lo": lo, "ci_hi": hi,
        "n_correct": nc, "n_total": nt, "elapsed_s": elapsed,
        "n_features": Xs_tr.shape[2], "n_classes": n_classes,
        "feature_cols": pre["feature_cols"],
    }
```

Listing 3: CNN-1D with global average pooling, Wilson CI, and early stopping.

4.3 Step 3 — The clean→contended operating-condition shift experiment

Define:

- Acc(clean → clean): model trained on clean-regime rows only (C1, C2, C4), evaluated on the held-out clean-regime test split.
- Acc(clean → contended): same clean-trained model, evaluated on the held-out contended-regime test split (C5).
- The operating-condition gap: $\Delta_{\text{op-shift}} = \text{Acc(clean} \rightarrow \text{clean)} - \text{Acc(clean} \rightarrow \text{contended)}$.

A larger $\Delta_{\text{op-shift}}$ means the contended regime introduces feature-distribution changes the clean-trained model cannot handle (most likely from half-duplex shadowing and ACK exchanges). The script reports $\Delta_{\text{op-shift}}$ for every non-skipped task; remember the per-task validity caveats from the prerequisites infobox.

```
def load_regime_split(task, regime, split):
    Xs = np.load(ARTIFACTS_DIR / task / f"Xs_{split}.npy")
    ys = np.load(ARTIFACTS_DIR / task / f"ys_{split}.npy")
    rs = np.load(ARTIFACTS_DIR / task / f"regs_{split}.npy")
    mask = (rs == regime)
    return Xs[mask], ys[mask]


for task, res in results.items():
    if res.get("skipped"):
        continue
    log.info(f"--- Op-shift: {task} ---")
    X_tr, y_tr = load_regime_split(task, "clean", "train")
    X_va, y_va = load_regime_split(task, "clean", "val")
    n_classes  = res["n_classes"]
    if len(X_tr) == 0:
        log.warning(f"  {task}: no clean-regime training rows --- "
                    f"skipping op-shift")
        res["op_shift"] = {"skipped": True, "reason": "no clean train"}
        continue

    # Train on clean rows only, with val-loss early stopping on clean val.
    # Note on the scaler: LH4-B fit StandardScaler on the full train split
    # (clean + contended), so the scaler statistics include a small
    # contribution from contended-regime rows. The leak is small (mean
    # and variance only) but real; refitting on clean-only training data
    # would require rebuilding the LH4-B artifacts and is left as a
```

```
    # documented trade-off.
    t0 = time.time()
    clean_model, _ = train_cnn(X_tr, y_tr, X_va, y_va, n_classes)
    log.info(f"  Trained on clean only ({len(X_tr)} sequences) "
             f"in {time.time() - t0:.1f}s")

    X_cc, y_cc = load_regime_split(task, "clean",     "test")
    X_cn, y_cn = load_regime_split(task, "contended", "test")
    acc_cc, nc_cc, nt_cc = accuracy(clean_model, X_cc, y_cc)
    acc_cn, nc_cn, nt_cn = accuracy(clean_model, X_cn, y_cn)
    delta = acc_cc - acc_cn if not np.isnan(acc_cn) else float("nan")
    log.info(f"  clean->clean = {acc_cc:.3f}  ({nc_cc}/{nt_cc})")
    log.info(f"  clean->contended = {acc_cn:.3f}  ({nc_cn}/{nt_cn})")
    log.info(f"  Delta = {delta:+.3f}")

    res["op_shift"] = {
        "acc_clean_clean":     acc_cc,
        "acc_clean_contended": acc_cn,
        "delta":               delta,
        "n_clean_test":        nt_cc, "n_contended_test": nt_cn,
        "clean_train_size":    int(len(X_tr)),
    }
    res["_clean_model_state"] = {k: v.cpu().clone()
                                 for k, v in clean_model.state_dict().
                                     items()}
```

Listing 4: Operating-condition shift evaluation reusing the canonical CNN.

Why On-Hardware Operating-Condition Shift, Not Sim-to-Real?

The framework deliberately measures distribution shift within hardware data because that is a more honest test of generalisation than synthetic-vs-real. The C5 regime introduces real RF-domain effects — half-duplex shadowing during ACK exchanges and multi-station collisions — that no simulator could reproduce faithfully. The gap measured here is therefore directly attributable to physical operating-condition changes, not to simulator-vs-hardware modelling artefacts. This positioning is also consistent with the final report's framing (Section IV.J): the operating-condition shift is the framework's capstone observation, and it is performed entirely on hardware data.

4.4 Step 4 — Mitigation: fine-tuning the final layer on contended data

Take the model trained on clean-regime data, freeze all layers except the final `Linear`, and fine-tune on a small fraction (10–20%) of the contended-regime training set. Measure the new accuracy on the contended-regime test set. Expected outcome: most of the operating-condition gap closes, demonstrating that the convolutional backbone learned useful representations but the classifier head was miscalibrated to the clean-regime feature distribution. The fine-tuning recovery is the framework-required outcome on the SF task: at least half of $\Delta_{\text{op-shift}}$ should be recovered. For `beacon` and `pkt`, fine-tuning has to teach the model classes it never saw during clean-only training, so the recovery there is closer to "how well does a small contended-regime fine-tune set bootstrap a new class" than "how much shift is recovered."

```
def finetune_last_layer(task, n_classes, clean_state, fraction=0.20,
```

```
                          epochs=40, batch=32, lr=1e-3, seed=42):
    torch.manual_seed(seed); np.random.seed(seed)
    model = CNN1D(results[task]["n_features"], n_classes).to(DEVICE)
    model.load_state_dict(clean_state)
    for p in model.parameters():            p.requires_grad = False
    for p in model.net[-1].parameters():    p.requires_grad = True

    X_ft, y_ft = load_regime_split(task, "contended", "train")
    if len(X_ft) == 0:
        log.warning(f"  {task}: no contended training rows; skipping fine-tune")
        return None
    rng = np.random.default_rng(seed=seed)
    n_keep = max(1, int(fraction * len(X_ft)))
    keep = rng.choice(len(X_ft), size=n_keep, replace=False)
    X_ft, y_ft = X_ft[keep], y_ft[keep]

    ds = TensorDataset(torch.from_numpy(X_ft).float(),
                       torch.from_numpy(y_ft).long())
    dl = DataLoader(ds, batch_size=batch, shuffle=True)
    opt = torch.optim.Adam([p for p in model.parameters() if p.requires_grad],
                           lr=lr)
    loss_fn = nn.CrossEntropyLoss()
    model.train()
    for _ in range(epochs):
        for xb, yb in dl:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            opt.zero_grad()
            loss_fn(model(xb), yb).backward()
            opt.step()

    X_ev, y_ev = load_regime_split(task, "contended", "test")
    acc_ft, nc, nt = accuracy(model, X_ev, y_ev)
    return {"acc": acc_ft, "n_correct": nc, "n_total": nt,
            "n_finetune_rows": int(n_keep), "fraction": fraction}


for task, res in results.items():
    if res.get("skipped") or res.get("op_shift", {}).get("skipped"):
        continue
    log.info(f"--- Fine-tune: {task} ---")
    state = res.pop("_clean_model_state")     # consume; don't dump to JSON
    ft = finetune_last_layer(task, res["n_classes"], state)
    if ft is None:
        continue
    res["finetune"] = ft
    op = res["op_shift"]
    recovered = ft["acc"] - op["acc_clean_contended"]
    log.info(f"  contended-finetune test acc = {ft['acc']:.3f}  "
             f"(was {op['acc_clean_contended']:.3f}, "
             f"recovered {recovered:+.3f})")
    res["finetune"]["recovered"] = recovered
```

4.5 Step 5 — Tabulate and write manifest

```
log.info("=== Op-shift summary ===")
log.info(f"{'Task':<8} {'CNN test':>10} {'cln->cln':>10} "
         f"{'cln->cnt':>10} {'Delta':>10} {'finetune':>10} {'recover':>10}
            ")
for task in ["sf", "mod", "beacon", "pkt"]:
    res = results[task]
    if res.get("skipped"):
        log.info(f"{task:<8} (skipped)  reason: {res['reason']}")
        continue
    op = res.get("op_shift", {})
    ft = res.get("finetune", {})
    log.info(f"{task:<8} {res['test_acc']:>10.3f} "
             f"{op.get('acc_clean_clean', float('nan')):>10.3f} "
             f"{op.get('acc_clean_contended', float('nan')):>10.3f} "
             f"{op.get('delta', float('nan')):>+10.3f} "
             f"{ft.get('acc', float('nan')):>10.3f} "
             f"{ft.get('recovered', float('nan')):>+10.3f}")

manifest = {
    "generated_at": datetime.now(timezone.utc).isoformat(),
    "run_stamp":    run_stamp,
    "log_archive":  str(log_archive),
    "device":       str(DEVICE),
    "window":       WINDOW,
    "results":      results,
}
with open(MANIFEST_PATH, "w", encoding="utf-8") as f:
    json.dump(manifest, f, indent=2, default=str)
log.info(f"Wrote {MANIFEST_PATH}")
log.info("=== cnn_train.py done ===")
```

5 Verification Checklist

- ☐ Sliding-window sequences built; grouping is `(run_id, beacon_id)` so per-station histories are not contaminated by adjacent stations.
- ☐ CNN-1D trained to completion on every task with ≥2 classes (`sf`, `beacon`, `pkt`; `mod` skipped with logged reason).
- ☐ Wilson 95% CI reported for each canonical CNN test accuracy.
- ☐ clean→clean and clean→contended accuracies recorded for every non-skipped task.
- ☐ Operating-condition shift $\Delta_{\text{op-shift}}$ tabulated and linked back to LH3-H Step 8 operating-condition shift findings.

Task	**clean→clean**	**clean→contended**	**$\Delta_{\text{op-shift}}$**	**finetune**	**Likely cause / caveat**
SF Detection					
Modulation	(skipped — only LoRa collected)				
Emitter ID					Beacon ID 0x02 only in contended; Delta is the new-class gap
Packet Type					ACK only in contended; Delta is the new-class gap

Table 1: Operating-condition shift table. Fill from the script's printed summary or from `cnn_manifest.json`. Each non-trivial row's "likely cause" should cite the specific distributional difference from LH3-H's Step 8 that explains the observed drop — typically half-duplex shadowing or ACK-row insertion.

- ☐ Fine-tuning experiment run; for the SF task it recovers $\geq$ half of $\Delta_{\text{op-shift}}$. For `beacon`/`pkt`, fine-tune is reported but interpretation is the new-class bootstrap caveat (per the prerequisites infobox), not standard op-shift recovery.
- ☐ `Code/AIML/artifacts/<task>/cnn.pt` saved per non-skipped task.
- ☐ `Code/AIML/cnn_manifest.json` written.
- ☐ `Code/AIML/Logs/cnn_train_<timestamp>.log` archived.

Lab Handout 4-F Deliverable

1. `Code/AIML/cnn_train.py`: the runnable script.
2. `Code/AIML/cnn_manifest.json`: per-task canonical accuracy with Wilson CI, op-shift triple (clean→clean, clean→contended, Δ), and fine-tune outcome.
3. `Code/AIML/artifacts/{sf,beacon,pkt}/cnn.pt`: canonical CNN state dicts.
4. Filled operating-condition shift table with likely-cause column citing LH3-H's Step 8 operating-condition shift analysis.
5. Fine-tuning result: clean→clean, clean→contended, and clean+contended-finetune accuracies on the same plot per non-skipped task.
6. One-page discussion: which task shows the largest $\Delta_{\text{op-shift}}$, why, and what additional contended-regime collection (longer C5b runs or more diverse station configurations) would most efficiently close it. Explicitly distinguish the SF op-shift result (a continuous distribution shift on a class set common to both regimes) from the `beacon`/`pkt` numbers (which measure how well a clean-only model can be bootstrapped to a brand-new class via final-layer fine-tuning).

7. `Code/AIML/Logs/cnn_train_<timestamp>.log`: archived audit trail.